

git

Test de repaso — parte 2: gestión de ramas

83 preguntas · 4 opciones, 1 correcta

Cómo usar este banco de preguntas

- Cada diapositiva contiene una pregunta con 4 opciones (A-D), solo 1 correcta.
- No incluye las respuestas: úsalo para practicar de cara al examen. TODAS las preguntas del examen saldrán de este banco de preguntas.

Bloque 1

Repaso y `git commit --amend`

Pregunta 1

¿Qué comando usarías para corregir el mensaje del último commit sin crear uno nuevo?

- A) `git commit -m "nuevo mensaje"`
- B) `git commit --amend -m "nuevo mensaje"`
- C) `git log --amend`
- D) `git add --amend`

Pregunta 2

¿Cuándo es seguro usar `git commit --amend` ?

- A) Siempre, no tiene ningún riesgo
- B) Solo si el commit todavía no se ha compartido con el resto del equipo
- C) Solo si es el primer commit del repositorio
- D) Nunca, está prohibido

Pregunta 3

¿A cuántos commits afecta `git commit --amend` ?

- A) A todos los commits de la rama actual
- B) Únicamente al último commit
- C) A los dos últimos commits
- D) A ningún commit, solo al mensaje mostrado en pantalla

Pregunta 4

(Repaso parte 1) ¿Qué comando usarías para pasar un fichero del *working directory* a la *staging area*?

- A) `git commit`
- B) `git add`
- C) `git init`
- D) `git log`

Pregunta 5

¿Qué comando muestra el historial de commits en una sola línea por commit?

- A) `git log --oneline`
- B) `git status --short`
- C) `git diff --oneline`
- D) `git branch --oneline`

Pregunta 6

Ejecutas `git commit --amend` sin usar `-m`, y se abre un editor de texto. ¿Qué está esperando git?

- A) Que confirmes el borrado del commit
- B) Que escribas (o edites) el mensaje del commit
- C) Que introduzcas tu contraseña
- D) Que elijas una rama

Bloque 2

¿Qué es **HEAD**?

Pregunta 7

¿Qué es `HEAD` en git?

- A) El primer commit del repositorio
- B) Un puntero que indica en qué commit/rama estás situado ahora mismo
- C) El nombre de la rama principal
- D) Un fichero de configuración

Pregunta 8

Normalmente, ¿a qué apunta `HEAD` ?

- A) A una rama, y esa rama apunta al commit
- B) Directamente a un fichero del proyecto
- C) Al commit más antiguo del repositorio
- D) A ningún sitio hasta que haces el primer commit

Pregunta 9

Haces un commit nuevo en la rama en la que estás. ¿Qué le pasa a `HEAD` ?

- A) Deja de existir hasta el próximo checkout
- B) Avanza junto con el nuevo commit
- C) Se mueve automáticamente a `main`
- D) No cambia nunca

Pregunta 10

Ejecutas `git checkout otra-rama`. ¿Qué le pasa a `HEAD` ?

- A) Desaparece
- B) Pasa a apuntar a `otra-rama` y a su último commit
- C) Se queda apuntando a la rama anterior
- D) Se convierte en un tag

Pregunta 11

En `git log --oneline` ves `(HEAD -> main)` junto a un commit. ¿Qué significa?

- A) Que ese commit se ha borrado
- B) Que estás situado en la rama `main`, sobre ese commit
- C) Que ese commit pertenece a otro repositorio
- D) Que `main` y `HEAD` son ramas distintas

Pregunta 12

¿Por qué es importante entender `HEAD` antes de usar `branch`, `checkout` y `merge` ?

A) No es necesario, son conceptos independientes

B) Porque los tres comandos trabajan moviendo o comparando con `HEAD`

C) Porque `HEAD` solo se usa con `git log`

D) Porque `HEAD` es un alias de `main`

Bloque 3

git branch

Pregunta 13

¿Qué comando lista las ramas de un repositorio?

- A) `git branch`
- B) `git log --branches`
- C) `git status --branches`
- D) `git checkout --list`

Pregunta 14

¿Qué comando crea una rama nueva llamada `feature/login` sin cambiarte a ella?

- A) `git checkout feature/login`
- B) `git branch feature/login`
- C) `git checkout -b feature/login`
- D) `git merge feature/login`

Pregunta 15

Quieres eliminar una rama que ya ha sido fusionada en `main`. ¿Qué comando usas?

- A) `git branch -d <rama>`
- B) `git branch -D <rama>`
- C) `git checkout -d <rama>`
- D) `git remove <rama>`

Pregunta 16

Intentas borrar con `git branch -d` una rama que tiene cambios sin fusionar. ¿Qué ocurre?

- A) Se borra igualmente sin avisar
- B) Git se niega y avisa de que la rama no está fusionada
- C) Se fusiona automáticamente antes de borrar
- D) Se borra también `main`

Pregunta 17

¿Qué opción fuerza el borrado de una rama aunque tenga cambios sin fusionar?

- A) `git branch -f`
- B) `git branch -D`
- C) `git branch --force-delete`
- D) `git branch -x`

Pregunta 18

Estando situado en la rama que quieres renombrar, ¿qué comando usas?

- A) `git branch -m <nuevo_nombre>`
- B) `git branch --rename <nuevo_nombre>`
- C) `git checkout -m <nuevo_nombre>`
- D) `git rename <nuevo_nombre>`

Pregunta 19

Quieres renombrar una rama distinta a la que tienes activa ahora mismo, sin moverte a ella. ¿Qué comando usas?

- A) `git branch -m <nuevo_nombre>`
- B) `git branch -m <nombre_actual> <nuevo_nombre>`
- C) `git checkout <nombre_actual> && git branch -m <nuevo_nombre>`
- D) No es posible sin cambiarte primero a esa rama

Pregunta 20

¿Qué representan las ramas en git, según la analogía vista en clase?

- A) Copias completas y separadas del repositorio, sin relación entre sí
- B) Líneas de tiempo o versiones paralelas de un mismo proyecto
- C) Ficheros de configuración
- D) Commits especiales que no se pueden fusionar

Bloque 4

Convenciones de nombres y estrategias de ramificación

Pregunta 21

¿Qué prefijo de rama se usa habitualmente para una funcionalidad nueva?

- A) `fix/`
- B) `feature/`
- C) `release/`
- D) `hotfix/`

Pregunta 22

¿Qué prefijo se usa habitualmente para una corrección de un fallo?

- A) feature/
- B) fix/
- C) docs/
- D) init/

Pregunta 23

¿Cuál de estas afirmaciones sobre la convención de nombres de ramas es correcta?

- A) Es una norma oficial y obligatoria de git
- B) Es una convención muy extendida, pero no es obligatoria
- C) Solo se puede usar en GitHub
- D) Git rechaza los nombres de rama que no sigan esa convención

Pregunta 24

En Git Flow, ¿qué dos ramas son permanentes?

- A) feature y fix
- B) main y develop
- C) staging y production
- D) origin y upstream

Pregunta 25

¿Qué workflow se basa en tener solo `main` (siempre desplegable) y ramas `feature/` cortas que se fusionan vía pull request?

- A) Git Flow
- B) GitHub Flow
- C) Trunk-based development
- D) SVN Flow

Pregunta 26

¿Qué caracteriza al *trunk-based development*?

- A) Ramas de larga duración por cada funcionalidad
- B) Trabajar casi siempre directo sobre `main`, con ramas muy cortas o *feature flags*
- C) Prohibición total de hacer commits en `main`
- D) Uso obligatorio de `git reset`

Bloque 5

Visualizar el historial

Pregunta 27

¿Qué comando muestra el historial de commits representando gráficamente cómo se relacionan las ramas?

- A) `git log --graph --oneline --all`
- B) `git branch --graph`
- C) `git status --graph`
- D) `git diff --all`

Pregunta 28

En la salida de `git log --graph`, ¿qué representa una línea que se bifurca en dos (`| \`)?

- A) Un error en el historial
- B) El punto donde se creó una rama nueva a partir de ese commit
- C) Un commit borrado
- D) Un conflicto sin resolver

Pregunta 29

¿Por qué es útil `git log --graph --oneline --all` cuando llevas varias ramas abiertas?

- A) Porque borra las ramas que no necesitas
- B) Porque permite ver de un vistazo cómo se relacionan todas las ramas y commits
- C) Porque es el único comando que muestra los mensajes de commit
- D) Porque fusiona automáticamente las ramas

Pregunta 30

¿Qué añade la opción `--all` en `git log --graph --oneline --all` respecto a no usarla?

- A) Muestra también los commits de todas las ramas, no solo la actual
- B) Muestra solo los últimos 10 commits
- C) Oculta los commits fusionados
- D) Muestra los ficheros ignorados

Bloque 6

`git checkout` — lo básico

Pregunta 31

¿Qué comando usarías para cambiar de la rama actual a `feature/login` ?

- A) `git branch feature/login`
- B) `git checkout feature/login`
- C) `git merge feature/login`
- D) `git log feature/login`

Pregunta 32

¿Qué hace `git checkout -b feature/login` ?

- A) Solo crea la rama, sin cambiarte a ella
- B) Solo te cambia a una rama que ya existe
- C) Crea la rama y te cambia a ella en un solo paso
- D) Borra la rama `feature/login`

Pregunta 33

¿Qué hace `git checkout index.html` sobre un fichero ya confirmado con cambios sin preparar?

- A) Restaura el fichero a su última versión confirmada, descartando esos cambios
- B) Confirma los cambios del fichero
- C) Borra el fichero
- D) Lo añade a la staging area

Pregunta 34

Según lo visto en clase, ¿qué comandos recomienda git actualmente en vez de `git checkout` para cambiar de rama y restaurar ficheros, respectivamente?

- A) `git switch` y `git restore`
- B) `git branch` y `git reset`
- C) `git move` y `git undo`
- D) `git change` y `git recover`

Pregunta 35

¿Qué comando te lleva de vuelta a la rama principal `main` ?

- A) `git checkout main`
- B) `git branch main`
- C) `git merge main`
- D) `git init main`

Pregunta 36

Quieres situarte sobre un commit antiguo concreto de tu historial usando su hash. ¿Qué comando usas?

- A) `git checkout <hash>`
- B) `git branch <hash>`
- C) `git add <hash>`
- D) `git status <hash>`

Pregunta 37

¿Sigue siendo `git checkout` un comando válido y muy usado en git, aunque existan alternativas más modernas?

- A) No, está obsoleto y da error
- B) Sí, sigue siendo muy popular y es fundamental entenderlo
- C) Solo funciona en versiones antiguas de git
- D) Solo sirve para restaurar ficheros, nunca para cambiar de rama

Bloque 7

Detached HEAD

Pregunta 38

¿Qué ocurre cuando haces `git checkout <hash_de_commit>` en vez de un nombre de rama?

- A) Git da un error y no lo permite
- B) `HEAD` pasa a apuntar directamente a ese commit, sin ninguna rama de por medio
- C) Se crea automáticamente una rama nueva
- D) Se borra la rama actual

Pregunta 39

¿Cómo se llama el estado en el que te encuentras tras hacer checkout directo a un hash de commit?

- A) *Orphan branch*
- B) *Detached HEAD*
- C) *Floating commit*
- D) *Loose HEAD*

Pregunta 40

Estando en detached HEAD, haces un commit nuevo. ¿Qué rama apunta a ese commit?

- A) `main` , automáticamente
- B) La última rama en la que estuviste antes
- C) Ninguna: solo `HEAD` apunta a él, de forma temporal
- D) Se crea una rama con el nombre del hash

Pregunta 41

Después de hacer un commit en detached HEAD, cambias a otra rama sin hacer nada más. ¿Qué pasa con ese commit?

- A) Se fusiona automáticamente en la rama a la que te cambias
- B) Se mueve a `main`
- C) Queda "huérfano": nada lo referencia y con el tiempo puede eliminarse (garbage collection)
- D) Aparece automáticamente en `git branch`

Pregunta 42

¿Cómo evitas perder un commit hecho en detached HEAD antes de cambiar de rama?

- A) Ejecutando `git status` varias veces
- B) Creando una rama nueva en ese punto con `git checkout -b <nombre>`
- C) Es imposible evitarlo
- D) Haciendo `git commit --amend`

Pregunta 43

¿Cuál de estos mensajes es el que muestra git al entrar en detached HEAD?

- A) "You are in 'detached HEAD' state..."
- B) "Fast-forward"
- C) "CONFLICT (content)"
- D) "nothing to commit, working tree clean"

Bloque 8

git checkout avanzado

Pregunta 44

¿Qué comando trae únicamente el fichero `styles.css` desde la rama `feature/x` a la rama actual, sin fusionar toda la rama?

- A) `git merge feature/x -- styles.css`
- B) `git checkout feature/x -- styles.css`
- C) `git branch feature/x styles.css`
- D) `git add feature/x styles.css`

Pregunta 45

Tras recuperar un fichero de otra rama con `git checkout <rama> -- <fichero>`, ¿en qué estado queda ese fichero en tu rama actual?

- A) Untracked
- B) Preparado (*staged*), listo para confirmar
- C) Ignorado
- D) Sin ningún cambio

Pregunta 46

¿Qué comando crea una rama nueva a partir de un commit específico (no del último de la rama actual)?

- A) `git checkout -b <rama> <hash>`
- B) `git branch --from <hash> <rama>`
- C) `git merge <hash> <rama>`
- D) `git log -b <rama> <hash>`

Pregunta 47

¿Para qué sirve recuperar un fichero suelto de otra rama con `git checkout <rama> -- <fichero>` ?

- A) Para fusionar automáticamente todos los cambios de esa rama
- B) Para traer solo ese fichero concreto, sin mezclar el resto de cambios de la rama
- C) Para borrar el fichero de la otra rama
- D) Para crear un conflicto a propósito

Pregunta 48

Si creas una rama con `git checkout -b experimento <hash>` y ese hash no es el último commit de ninguna rama, ¿qué logras?

- A) Nada, git lo rechaza
- B) Una rama nueva que arranca justo en ese commit concreto del historial
- C) Fusionar ese commit en `main` automáticamente
- D) Borrar todos los commits posteriores

Bloque 9

Fast-forward vs merge de tres vías

Pregunta 49

¿Cuándo hace git un merge "fast-forward"?

- A) Siempre, en cualquier fusión
- B) Cuando la rama destino (ej. `main`) no ha cambiado desde que se creó la rama que fusionas
- C) Solo cuando hay un conflicto
- D) Solo si usas la opción `--fast`

Pregunta 50

En un merge fast-forward, ¿qué hace git realmente?

- A) Crea un commit de fusión nuevo
- B) Simplemente mueve el puntero de la rama destino hasta el último commit de la otra rama
- C) Combina línea a línea todos los ficheros
- D) Borra el historial anterior

Pregunta 51

¿Cuándo se produce un merge de tres vías (recursive/ort)?

- A) Cuando la rama destino ha cambiado mientras tanto y hace falta combinar ambos historiales
- B) Cuando las dos ramas están completamente vacías
- C) Solo si se usa `--recursive` explícitamente
- D) Nunca, git siempre hace fast-forward

Pregunta 52

¿Qué genera un merge de tres vías que un fast-forward no genera?

- A) Un fichero `.gitignore`
- B) Un nuevo commit de fusión que combina ambos historiales
- C) Una rama nueva automáticamente
- D) Un conflicto obligatoriamente

Pregunta 53

¿Un merge de tres vías implica siempre que hay un conflicto?

- A) Sí, siempre
- B) No, puede combinarse automáticamente sin conflicto si los cambios no se solapan
- C) Solo si se usa `--force`
- D) Solo en la primera fusión del repositorio

Pregunta 54

En `git log --graph`, ¿cómo se distingue visualmente un merge de tres vías de un fast-forward?

- A) El fast-forward no genera bifurcación ni commit de fusión; el merge de tres vías sí muestra líneas que confluyen en un commit
- B) No hay ninguna diferencia visual
- C) El fast-forward siempre aparece en rojo
- D) El merge de tres vías nunca aparece en el log

Bloque 10

git merge y conflictos

Pregunta 55

¿Qué hace el comando `git merge <rama>` ?

- A) Borra la rama indicada
- B) Combina los cambios de esa rama con la rama en la que estás situado actualmente
- C) Crea una rama nueva
- D) Sube los cambios a un servidor remoto

Pregunta 56

Antes de fusionar `feature/x` en `main`, ¿qué paso previo es necesario?

- A) Estar situado en `main` (`git checkout main`)
- B) Borrar `feature/x`
- C) Hacer `git init` de nuevo
- D) Ninguno, se puede fusionar desde cualquier rama

Pregunta 57

¿Qué mensaje de salida indica que la fusión ha generado un conflicto que debes resolver tú?

- A) "Fast-forward"
- B) "Merge made by the 'ort' strategy"
- C) "CONFLICT (content): Merge conflict in <fichero>"
- D) "nothing to commit"

Pregunta 58

Dentro de un fichero en conflicto, ¿qué indica la línea `<<<<<< HEAD` ?

- A) El final del conflicto
- B) El inicio del contenido de la rama en la que estás (`HEAD`)
- C) El inicio del contenido de la otra rama
- D) Un error de sintaxis del fichero

Pregunta 59

¿Qué separa el contenido de "tu rama" del contenido de "la otra rama" dentro de un bloque de conflicto?

- A) =====
- B) ----
- C) ###
- D) >>>

Pregunta 60

¿Qué indica la línea `>>>>>> feature/login` al final de un bloque de conflicto?

- A) El inicio del conflicto
- B) El final del contenido que venía de la rama `feature/login`
- C) Que el conflicto se ha resuelto automáticamente
- D) Que hay que borrar el fichero completo

Pregunta 61

Después de decidir con qué contenido te quedas en un conflicto, ¿qué debes hacer antes de confirmar la fusión?

- A) Eliminar los marcadores (<<<<<< , ===== , >>>>>>) del fichero
- B) Dejar los marcadores tal cual, git los ignora
- C) Borrar el fichero completo
- D) Ejecutar `git init` de nuevo

Pregunta 62

¿Qué comandos cierran una fusión después de resolver un conflicto a mano?

- A) `git status` y `git log`
- B) `git add .` y `git commit`
- C) `git branch -d` y `git checkout`
- D) `git revert` y `git reset`

Bloque 11

```
git reset
```

Pregunta 63

¿Para qué se usa `git reset`, en términos generales?

- A) Para deshacer cambios moviendo hacia atrás el puntero de la rama actual
- B) Para crear una rama nueva
- C) Para visualizar el historial
- D) Para resolver conflictos automáticamente

Pregunta 64

¿Qué modo de `git reset` mueve el puntero pero conserva los cambios en la staging area?

- A) `--hard`
- B) `--mixed`
- C) `--soft`
- D) `--keep`

Pregunta 65

¿Cuál es el modo por defecto de `git reset` si no se indica ninguna opción?

- A) `--soft`
- B) `--hard`
- C) `--mixed`
- D) `--none`

Pregunta 66

¿Qué hace `git reset --hard` ?

- A) Mueve el puntero y borra los cambios por completo, sin conservarlos
- B) Solo cambia el mensaje del último commit
- C) Crea un nuevo commit que deshace el anterior
- D) Solo afecta a ficheros ignorados

Pregunta 67

¿Por qué es peligroso usar `git reset` sobre commits que ya han sido compartidos con el resto del equipo?

- A) No es peligroso en ningún caso
- B) Porque reescribe el historial y puede romper el historial de quien ya lo había descargado
- C) Porque borra automáticamente el repositorio remoto
- D) Porque cambia el nombre de usuario configurado

Pregunta 68

Según lo visto en clase, ¿cuál es la política sobre el uso de `git reset` en la asignatura?

- A) Se recomienda usarlo siempre que sea posible
- B) Está prohibido su uso en la asignatura
- C) Solo se puede usar los viernes
- D) Solo lo puede usar el profesor

Bloque 12

git revert

Pregunta 69

¿Qué hace `git revert <hash>` ?

- A) Borra ese commit del historial por completo
- B) Crea un nuevo commit que deshace los cambios de ese commit, sin eliminarlo del historial
- C) Cambia el mensaje de ese commit
- D) Fusiona ese commit en otra rama

Pregunta 70

¿Qué diferencia principal hay entre `git reset` y `git revert`?

- A) Ninguna, son sinónimos
- B) `reset` reescribe el historial; `revert` añade un commit nuevo que deshace cambios, sin tocar el historial existente
- C) `revert` solo funciona en `main`
- D) `reset` es más seguro que `revert`

Pregunta 71

¿Por qué es más seguro usar `git revert` que `git reset` cuando el commit ya se ha compartido con el equipo?

- A) Porque `revert` no cambia nada
- B) Porque `revert` no reescribe el historial existente, solo añade uno nuevo
- C) Porque `revert` borra el repositorio remoto
- D) No hay diferencia de seguridad entre ambos

Pregunta 72

Tras hacer `git revert abc1234`, ¿sigue apareciendo el commit `abc1234` en `git log`?

- A) No, desaparece
- B) Sí, sigue en el historial, junto con el nuevo commit que lo deshace
- C) Solo si se usa `--hard`
- D) Solo si se hace un `git init` nuevo

Pregunta 73

¿Qué necesitas indicarle a `git revert` para que sepa qué commit deshacer?

- A) El nombre de una rama
- B) El hash del commit que quieres deshacer
- C) El nombre de usuario del autor
- D) Nada, deshace automáticamente el último commit siempre

Pregunta 74

Si haces `git revert` sobre un commit y luego haces `git revert` sobre ese mismo commit de revert, ¿qué consigues aproximadamente?

- A) Borrar todo el historial
- B) Volver a un estado parecido al que había antes del primer revert
- C) Un error, no se puede revertir un revert
- D) Fusionar dos ramas

Bloque 13

Preguntas de escenario

Pregunta 75

Estás en `main`, con un historial estable. Creas una rama, haces 2 commits en ella y nadie más ha tocado `main` mientras tanto. Al fusionarla, ¿qué tipo de merge esperas?

- A) Fast-forward
- B) Merge de tres vías con conflicto seguro
- C) Ninguno, hay que borrar `main` primero
- D) `git reset` obligatorio

Pregunta 76

Mismo escenario, pero mientras trabajabas en tu rama, un compañero fusionó otro commit en `main`. Al fusionar tu rama ahora, ¿qué tipo de merge es más probable?

- A) Fast-forward
- B) Merge de tres vías (recursive/ort)
- C) No se puede fusionar nunca en ese caso
- D) `git revert` automático

Pregunta 77

Dos ramas distintas modifican la misma línea del mismo fichero de formas diferentes. Al fusionar la segunda rama, ¿qué es lo más probable que ocurra?

- A) Git elige automáticamente la versión más reciente sin avisar
- B) Se produce un conflicto que hay que resolver a mano
- C) Se borra el fichero
- D) Se genera un fast-forward

Pregunta 78

Quieres deshacer un commit que ya se subió y que ha visto todo el equipo. Según la política de la asignatura, ¿qué comando usas?

- A) `git reset --hard`
- B) `git revert`
- C) `git branch -D`
- D) `git checkout --hard`

Pregunta 79

Haces checkout de un commit antiguo por curiosidad, sin crear rama, miras el código y vuelves a `main` sin hacer ningún commit nuevo ahí. ¿Corres riesgo de perder algo?

- A) Sí, siempre se pierde algo al hacer checkout de un commit
- B) No, si no has hecho ningún commit en detached HEAD no hay nada que perder
- C) Sí, se borra automáticamente `main`
- D) Solo si usas `git log` a la vez

Pregunta 80

Necesitas saber en qué rama y en qué commit estás situado justo ahora. ¿Qué concepto/comando te lo indica?

- A) `.gitignore`
- B) `HEAD` (visible, por ejemplo, con `git log --oneline`)
- C) `git diff`
- D) `git config --list`

Pregunta 81

Vas a crear una rama para corregir un error urgente en producción, siguiendo la convención vista en clase. ¿Qué nombre le pondrías?

- A) `hotfix/bug-urgente` o `fix/bug-urgente`
- B) `main2`
- C) `prueba`
- D) `temporal`

Pregunta 82

Terminas de fusionar una rama en `main` y ya no la necesitas más. Según las buenas prácticas vistas, ¿qué deberías hacer?

- A) Dejarla ahí para siempre, por si acaso
- B) Borrarla con `git branch -d` para no acumular ramas muertas
- C) Renombrarla a `main`
- D) Convertirla en detached HEAD

Pregunta 83

Un compañero te dice que hizo cambios importantes en detached HEAD y luego cambió de rama sin crear una rama de rescate. ¿Qué le ha pasado probablemente a esos commits?

- A) Se han fusionado automáticamente en `main`
- B) Han quedado huérfanos y es probable que se pierdan (garbage collection)
- C) Se han convertido en tags automáticamente
- D) No ha pasado nada, siguen siendo parte de `main`

¡Suerte con el repaso!